

ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Тест-дизайн и тестовая документация

Модуль 2 · Лекция 3 · Тест-дизайн

Ольга Лихачева · Lekton

4 курс

17 сентября · Четверг 13:30–15:00

План лекции

01 Почему нельзя протестировать всё

03 Классификация техник тест-дизайна

05 Граничные значения: определение и методика построения

07 От техники к тест-кейсу

09 Пример из практики и практика 2

02 Что такое тест-дизайн

04 Классы эквивалентности: определение и методика построения

06 Попарное тестирование: комбинации и инструменты

08 Тестовая документация: стратегия, план, чек-листы, тест-кейсы

Утверждение: нельзя протестировать всё

Принцип ISTQB №2: исчерпывающее тестирование невозможно.

Пример из реальных тестов

```
POST /api/v1/clients/create-individual
```

- Один запрос содержит множество полей
- Поля делятся на обязательные и необязательные
- У каждого поля свой формат и ограничения

```
POST /api/v1/clients/create-individual
{
  "contacts": {
    "email": "user@example.com",
    "phone": "+79161234567",
    "notification_phone": null
  },
  "individual": {
    "demographic": {
      "birth_date": "1981-11-13",
      "gender": "Male",
      "marital_status": "Single"
    }
  },
  "tax": { "tax_number": "123456789012" }
}
```

Структура запроса создания клиента

Полный перебор полей и их значений невозможен — необходим системный отбор проверок.

Что такое тест-дизайн

Тест-дизайн

— процесс выбора и структурирования тестовых условий и тест-кейсов на основе анализа целей тестирования (ISTQB CTFL, раздел «Test design»).

Тестовое условие — что проверить (аспект, свойство, ситуация).

Тестовый кейс — как проверить: конкретные входные данные, шаги и ожидаемый результат.

Вход: требования и модель системы.

Выход: набор условий и кейсов.

Критерий: максимум обнаруженных дефектов при разумных затратах времени и ресурсов.

Парадокс пестицидов

(B. Beizer, «Software Testing Techniques»): многократный прогон одного и того же набора тестов по неизменённому коду снижает его способность находить новые дефекты — набор нужно периодически пересматривать и обновлять.

Классификация техник тест-дизайна

Техники разделяются по тому, на что они опираются: спецификация, структура кода или опыт тестировщика.

■ Specification-based (чёрный ящик)

Источник — спецификация и/или требования

КЭ, ГЗ, таблицы решений, state transition, попарное тестирование.

Пример: форма регистрации с полем — email без «@», с пробелом, длиной 200 символов.

■ Structure-based (белый ящик)

Источник — структура кода: ветви, условия, пути. Измеряем покрытие кода

Пример: функция с тремя ветками if — проверяем, что каждая выполнена хотя бы раз.

● Experience-based (эвристические)

Источник — знание и опыт тестировщика

Error guessing, exploratory, checklist-based

Пример: «что будет если?»

Сегодня в фокусе — спецификационные (чёрный ящик) техники: классы эквивалентности (КЭ), граничные значения (ГЗ) и попарное тестирование.

МОДУЛЬ 2 · ЛЕКЦИЯ 3

Техники тест-дизайна

Классы эквивалентности: определение

Класс эквивалентности (equivalence partition) — множество входных значений, которые система обрабатывает **одинаково**: внутри класса поведение не различается, между классами — различается.

Достаточно одного представителя класса

По определению все значения внутри одного КЭ обрабатываются одинаково.

Негативное тестирование

Позитивные и негативные классы выделяем **раздельно**: система должна уметь корректно реагировать на данные из негативных классов

Пример: поле email в POST /api/v1/clients/create-individual

✓ Позитивный класс: формат корректен → user@example.com

✗ Негативные классы: без @ · с пробелом · длиной > 254

Классы эквивалентности: методика

Шаги

1. Выделить классы эквивалентности по каждому полю.
2. Создать тест-кейс на каждый класс — минимум один представитель.

Отдельно

Негативные классы выделяем **отдельно**.

Пример: обязательные поля заведения клиента

- ✓ email: формат корректен → user@example.com
- ✓ phone: +7XXXXXXXXXX → +79161234567
- ✓ birth_date: прошлое, возраст ≥ 18 → 1981-11-13
- ✓ gender: значение из списка → Male
- ✓ tax_number: верная длина и только цифры

- ✗ без @ · с пробелом
- ✗ 10 цифр · буквы
- ✗ будущая дата · возраст < 18
- ✗ пусто · значение не из списка
- ✗ буквы · неверная длина

Классические негативные классы: значение **ниже/выше диапазона**, **не из списка**, **неверный формат**, **пустое значение**.

Правило: несколько невалидных значений **нельзя** объединять в один тест.

Классы эквивалентности: пример — создание клиента

Требование: POST /api/v1/clients/create-individual — валидация полей запроса

`contacts.email`

- ✓ Валидный: user@example.com
- ✗ Без @ · с пробелом · длина > 254

`individual.demographic.birth_date`

- ✓ Валидный: 1981-11-13
- ✗ Будущая дата · неверный формат · возраст < 18

`individual.demographic.gender`

- ✓ Валидный: Male
- ✗ Пусто · значение не из списка

`contacts.phone`

- ✓ Валидный: +79161234567
- ✗ 10 цифр · буквы · пусто

Итог: **N классов** → **N тест-кейсов** — по одному представителю на класс.

Один невалидный класс = один тест: иначе не локализуем причину.

Классы эквивалентности: типы классов



Непрерывный диапазон

Возраст по `birth_date`, например
18–120: 1 валидный + 2
невалидных (ниже и выше)



Дискретный набор

`gender / marital_status`: 1
валидный + невалидные (не из
списка, пусто)



Одно значение

`tax_number` фиксированного
формата: валидный класс —
само это значение



Набор значений

Если список мал — по
представителю на каждое
значение (например, `gender`)



Выходные значения

Классы и по ответам API: успех
200, ошибка валидации 400,
отказ 403

Классы эквивалентности: частые ошибки



Лишние тесты в классе



Смешивание невалидных полей



Пересечение классов



Игнорирование дизайна

Пересечения классов приходятся на границы классов; именно там чаще всего выявляются дефекты.

Позитивные и негативные проверки

Правило: сначала — **позитивный** тест, затем — **негативный**.

Порядок обоснован тем, что негативная проверка информативна только при подтверждённом основном сценарии: если корректный запрос не выполняется, ответ `400` не подтверждает наличие валидации.

Сквозной пример: создание клиента

Позитивный: `POST /api/v1/clients/create-individual` со всеми корректными полями → `200`, клиент создан.

Негативный: email без «@» или возраст 17 → `400`, клиент не создан.

Граничные значения: определение

Граничные значения (boundary value analysis, BVA) — техника тест-дизайна, при которой тестовые данные выбираются на границах классов эквивалентности и рядом с ними, а не внутри классов.

Большинство дефектов концентрируется на границах классов эквивалентности.

ON — граница

Минимальное или максимальное допустимое значение класса.

OFF — за границей

Значение непосредственно за границей (off-by-one) — соседний класс.

Причина: ошибки сравнений ($\geq$ вместо $>$, $\leq$ вместо $<$) и «off-by-one» при работе с диапазонами и счётчиками.

BVA — «ошибко-ориентированное» **дополнение** к классам эквивалентности, а не замена.

В примере заведения клиента граничными значениями являются **возраст ровно 18** по `birth_date` и **email длиной 254** символа.

Граничные значения: методика

1. Выделить классы эквивалентности

2. Найти границы каждого класса

3. Для каждого ГЗ — три точки: на границе, ниже, выше (on/off)

Единица измерения задаёт шаг

int: 16 → {15, 16, 17}

деньги: 5.00 → {4.99, 5.00, 5.01}

строка: max 255 → {254, 255, 256} символов

Отсутствие дублирования

Если точка выше/ниже границы уже попадает в соседний класс — не дублировать.

birth_date · возраст ≥ 18

{17, 18, 19} — ON: 18; OFF: 17; контрольная точка валидной зоны: 19

phone · +7XXXXXXXXXX

{10, 11, 12} цифр — ON: 11, OFF: 10 и 12

Для строгих границ иногда хватает двух точек (внутри/снаружи), но **тройка надёжнее**.

Граничные значения: пример

Тройки точек для сквозного примера клиента:

```
birth_date · возраст ≥ 18:  
{17, 18, 19}
```

```
email · max 254 символа:  
{253, 254, 255}
```

```
phone · +7XXXXXXXXXX:  
{10, 11, 12} цифр
```

```
display_name · max 255:  
{254, 255, 256}
```

Тот же принцип в нейтральных примерах: rate limit 100 req/s → {99, 100, 101}; VARCHAR(255) → {254, 255, 256} символов.

Граничные значения: структура и значение

BVA применяется и к значению, и к структуре данных.

Границы по значению — возраст клиента ≥ 18 : {17, 18, 19}.

Границы по длине (это структура) — phone +7XXXXXXXXXX: {10, 11, 12} цифр.

Структурные границы запроса: пустой `individual` (нет обязательных полей), пустой `contacts[]`, `null` в необязательном поле, число элементов массива.

Для невалидных классов границы используем тогда, когда ожидаем **разное поведение при обработке ошибки**.

Классы эквивалентности + Граничные значения — базовая пара

Классы эквивалентности

Задают группы значений с одинаковой обработкой.

Граничные значения

Уточняют, что происходит ровно на границе классов эквивалентности.

Применяются **всегда**, для любого объекта тестирования.

Итоговый набор: **представитель класса + границы каждого класса**.

Пример (клиент): email — представитель `user@example.com` + границы {253, 254, 255}; возраст — представитель 30 + границы {17, 18, 19}.

Обязательно комбинировать с другими техниками тест-дизайна

Попарное тестирование: определение

Попарное тестирование (pairwise testing, all-pairs) — техника тест-дизайна, при которой набор тестов строится так, чтобы каждая пара значений любых двух параметров встретилась хотя бы в одном тест-кейсе.

Цель — покрыть все попарные комбинации (покрытие всех пар, 2-wise coverage) при кратном сокращении числа тестов.
Основание — эмпирическое наблюдение: большинство дефектов вызывается взаимодействием не более двух факторов.

Итог для модели из семи параметров: полный перебор — 1296 комбинаций, попарный набор — около 16 строк, сокращение примерно в 81 раз.

Ограничение: покрытие всех пар не гарантирует полноту — взаимодействие трёх и более параметров остаётся непроверенным.

Подробный расчёт — на слайде «Попарное тестирование: расчёт на примере».

Попарное тестирование: эмпирические основания

AT&T

AT&T Bell Labs, 1992

P. Brownlie. Попарный набор (≈ 1000 кейсов) обнаружил больше дефектов, чем исходный план регрессии из ≈ 1500 тестов, при существенно меньших затратах. Точная величина выигрыша зависит от проекта.

NIST

NIST, Kuhn & Wallace, 2001

Анализ отзывов медицинских устройств за 15 лет: подавляющее большинство отказов (обычно цитируется $\approx 97-98\%$) воспроизводится взаимодействием не более двух факторов.

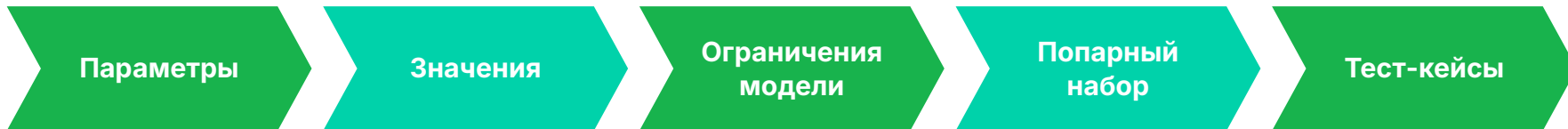
Mozilla

Kuhn & Reilly, 2002

Анализ баг-трекера Mozilla: набор с полным попарным покрытием (2-wise) выявил бы порядка 70–76% зарегистрированных дефектов.

Это эмпирические данные, а не доказанный закон. Покрытие всех пар (2-wise) не гарантирует полноты: критичные взаимодействия трёх и более факторов остаются непроверенными, а важность комбинаций неравномерна.

Попарное тестирование: методика



Шаги построения попарного набора

1. Выделить параметры и их значения из требований к объекту тестирования.
2. Задать ограничения модели — несовместимые сочетания значений.
3. Сгенерировать набор строк, покрывающий каждую пару значений любых двух параметров.
4. Спроецировать каждую строку набора в тест-кейс: одна строка — один тест-кейс.

Сквозной пример — модель клиента `create-individual`, 7 параметров

Полный перебор — 1296 комбинаций, попарный набор — ≈ 16 строк (расчёт — на следующем слайде). Каждая строка набора соответствует одному тест-кейсу.

Попарное тестирование: расчёт на примере

Модель клиента `create-individual` — 7 параметров

`gender` — 3 · `marital_status` — 4 · `age_group` — 3

`tax_number` — 3 · `external_id` — 2 · `display_name` — 2 · `phone` — 3

1296

полный перебор

$3 \times 4 \times 3 \times 3 \times 2 \times 2 \times 3$

170

пар значений

на $C(7,2) = 21$ пару параметров

≈ 81×

сокращение:

$1296 \rightarrow \approx 16$ строк

Расчёт шаг за шагом

1. Полный перебор: $3 \times 4 \times 3 \times 3 \times 2 \times 2 \times 3 = 1296$ комбинаций.
2. Число пар параметров: $C(7,2) = 21$.
3. Число пар значений по всем этим парам — **170**.
4. Нижняя граница: одна строка покрывает 21 пару, значит $\lceil 170 / 21 \rceil = 9$ строк.
5. Практический набор инструмента из-за конфликтов между парами — **≈ 16** строк.
6. Выигрыш: $1296 / 16 \approx 81\times$ при покрытии всех пар (2-wise).

Попарное тестирование: число пар значений

Модель клиента create-individual: 7 параметров, количества значений 3 + 4 + 3 + 3 + 2 + 2 + 3

1. Число пар параметров

$$C(n, 2) = n \times (n - 1) / 2$$

Порядок в паре не важен: (gender, phone) и (phone, gender) это одно и то же сочетание, покрыть его достаточно один раз.

Общий вид: $C(n, k) = n! / (k! \times (n - k)!)$

$$n = 7 \rightarrow 7 \times 6 / 2 = 21$$

2. Число пар значений

$$\sum k(i) \times k(j) \text{ по всем парам } i < j$$

$k(i)$ это количество значений параметра i . Произведения складываются по всем парам параметров.

Свёрнутая форма: $((\sum k(i))^2 - \sum k(i)^2) / 2$

$$\sum k(i) = 20, \sum k(i)^2 = 60 \rightarrow (400 - 60) / 2 = 170$$

Различайте: 21 это число **пар параметров**, оно зависит только от количества параметров. 170 это число **пар значений**, оно зависит ещё и от количества значений у каждого параметра.

Число пар значений: перебор

Параметр	Знач.	Произведения по парам с остальными	Сумма
gender	3	$3 \times 4 = 12 \cdot 3 \times 3 = 9 \cdot 3 \times 3 = 9 \cdot 3 \times 2 = 6 \cdot 3 \times 2 = 6 \cdot 3 \times 3 = 9$	51
marital_status	4	$4 \times 3 = 12 \cdot 4 \times 3 = 12 \cdot 4 \times 2 = 8 \cdot 4 \times 2 = 8 \cdot 4 \times 3 = 12$	52
age_group	3	$3 \times 3 = 9 \cdot 3 \times 2 = 6 \cdot 3 \times 2 = 6 \cdot 3 \times 3 = 9$	30
tax_number	3	$3 \times 2 = 6 \cdot 3 \times 2 = 6 \cdot 3 \times 3 = 9$	21
external_id	2	$2 \times 2 = 4 \cdot 2 \times 3 = 6$	10
display_name	2	$2 \times 3 = 6$	6
phone	3	новых пар нет, см. строки выше	0

Итог по строкам
 $51 + 52 + 30 + 21 + 10 + 6 = 170$

Проверка формулой
 $(20^2 - 60) / 2 = 170$

Нижняя граница строк
 $\text{ceil}(170 / 21) = 9$

Одна строка набора содержит $C(7, 2) = 21$ пар, поэтому корректная 2-wise-таблица обязана покрыть все 170 пар значений.

Попарное тестирование: ортогональные массивы и жадные алгоритмы

Ортогональные массивы

Математическая конструкция, в которой каждая пара значений встречается одинаковое число раз.

Обозначение $L_9(3^5)$ — 9 строк для пяти параметров по три значения; известны и более крупные массивы, например $L_{64}(8^2 \cdot 4^3)$. Размеры массива заданы заранее и «квантованы», поэтому модель 3/4/3/3/2/2/3 аккуратно в готовый массив не раскладывается.

Жадные алгоритмы (AllPairs)

Набор строится прямым покрытием пар: строки добавляются, пока не закрыты все пары значений. Строк обычно примерно на четверть меньше, чем в ортогональном массиве, и алгоритм применим к произвольной модели. Если значение в строке на покрытие не влияет, оно помечается знаком «~» — «значение несущественно».

Оба подхода приводят к сопоставимому по размеру набору; конкретные числа для модели клиента приведены в расчёте на предыдущем слайде.

Инструменты построения попарных наборов

PICT (Microsoft)

Вход — текстовый файл модели (параметры и их значения);
выход — набор строк, по одной на тест-кейс. Инструмент с
открытым кодом, кроссплатформенный.

```
gender: Male, Female, Other
marital_status: Single, Married, Divorced, Widowed
age_group: under18, 18-65, over65
tax_number: valid, empty, invalid
external_id: set, unset
display_name: set, unset
phone: set, unset, invalid
```

```
$ pict model.txt > cases.txt
```

Жадные алгоритмы: AllPairs (James Bach)

Тот же принцип входа и выхода; распространяется как
инструмент с открытым кодом.

```
$ allpairs model.txt > cases.txt
```

Ограничения модели: несовместимые сочетания задаются
правилами, например IF [age_group] = "<18" THEN
[marital_status] <> "Married". Инструмент исключает такие
сочетания из генерации.

Ограничение инструментов: они работают только с заданной
моделью и не выводят параметры, ограничения и критичные
сочетания из требований. Каталог инструментов —
pairwise.org (PICT, AllPairs и другие).

Попарное тестирование: ограничения и область применения

Ограничения

Покрытие всех пар (2-wise) не покрывает взаимодействия трёх и более параметров; обобщение — покрытие всех n-наборов (n-wise), но с ростом n растёт и число строк.

Несовместимые сочетания значений задаются **ограничениями модели**: например, `under18 + Married` исключается из набора.

Критичные сочетания, известные команде, добавляются **вручную** — генератор их не выводит.

Техника **не заменяет анализ требований**, а дополняет его риск-ориентированно.

Область применения

Параметров много, зависимости между ними неизвестны или слабо формализованы.

Полный перебор недопустим по срокам, требуется приемлемое соотношение затрат и эффекта.

Для модели клиента `create-individual`: **1296** полных комбинаций → **≈ 16** строк, покрывающих все пары.

Техники на практике: сервис Authorization

Классы эквивалентности

Статус карты: ACTIVE / INACTIVE / BLOCKED / EXPIRED — четыре класса.

Сумма транзакции: допустимая / нулевая / отрицательная / превышающая доступный баланс.

Граничные значения

dailyLimit: на границе — сумма равна лимиту; за границей — лимит + 1.

monthlyLimit: на границе — сумма равна лимиту; за границей — лимит + 1.

availableBalance: на границе — сумма равна балансу; за границей — баланс - 0.01.

expiryDate (ММYY): на границе — текущий месяц; за границей — предыдущий.

Результат применения техник оформляется как **тестовая документация** — это задание практики 2.

От учебного примера к практике

Учебный пример — клиент

Заведение клиента `POST /api/v1/clients/create-individual` — сквозная иллюстрация техник на лекции: классы, границы, попарное тестирование.

Объект практики — Authorization

На практике 2 те же техники применяются к сервису Authorization и модели Card: статусы карты, лимиты, баланс, срок действия.

Техника не зависит от предметной области: меняется объект тестирования, порядок применения остаётся тем же.

От техники к тест-кейсу

Правила перехода

Классы эквивалентности: каждый класс → один тест-кейс на представителя класса

Граничные значения: каждая граница → три точки — на границе, ниже и выше

Попарное тестирование: каждая строка набора → отдельный тест-кейс

Каждый тест-кейс фиксирует: предусловие → шаги → ожидаемый результат

Пример

Класс «статус карты BLOCKED» → POST /auth:
карта BLOCKED → ответ DECLINED с кодом причины

Граница «сумма равна dailyLimit» → POST /auth: сумма = dailyLimit → ответ APPROVED

Граница «сумма превышает баланс на 0.01» → POST /auth → ответ DECLINED

МОДУЛЬ 2 · ЛЕКЦИЯ 3

Тестовая документация

Тестовая документация: иерархия

Тестовая политика и стратегия

Стратегия — основания и область

Уровень организации или проекта. Отвечает: почему выбран такой подход и что входит в область тестирования, а что — вне неё.

Тестовый план

Чек-листы и тест-кейсы

План — способ, роли, сроки

Уровень проекта, релиза, итерации. Декомпозирует стратегию на объём работ, подход, ресурсы, критерии, риски.

Чек-листы и тест-кейсы — содержание проверок

Чек-лист задаёт перечень проверок; тест-кейс фиксирует данные, шаги и ожидаемый результат.

Отчёты и дефекты — результаты

Протоколы прогонов и отчёты о дефектах; служат основанием для корректировки плана и стратегии.

Сквозной пример — создание клиента, `POST /api/v1/clients/create-individual`

Стратегия: создание клиента отнесено к критическому пути: без клиента невозможны открытие счёта и транзакции.

План: валидация обязательных полей `contacts / individual / tax`; критерий выхода — отсутствие открытых дефектов критического уровня.

Чек-лист: обязательные поля · необязательные поля · форматы значений · дубли `external_id`.

Тест-кейс: email длиной 255 символов → HTTP 400 с кодом валидации.

Отчёт и дефект: 3 из 4 проверок пройдены; дефект: email длиной 255 символов не отклоняется.

Тестовая стратегия

Тестовая стратегия (test strategy) — долгосрочное описание подхода к тестированию на уровне организации или программы проекта. Задаёт уровни и виды тестирования, принципы приоритизации, а также область тестирования и то, что осознанно оставлено вне области тестирования.

Уровни применения

Тестовая политика — организационный уровень; общие цели и правила.

Тестовая стратегия — уровень программы или проекта; подход.

Тестовый план — конкретная работа: способ, роли, сроки.

Назначение

Единый критерий решений: что тестируется в первую очередь, где останавливаемся, от чего отказываемся. Приоритет определяется риском: объём следует за риском, а не за удобством.

Подходы к организации (ISTQB): часть 1

Аналитический — проверки выводятся из формального анализа, прежде всего анализа рисков.

Риск-ориентированный — приоритет по вероятности и последствиям дефектов.

Модельный — строится модель системы (состояния, данные, потоки), проверки выводятся из неё.

Подходы к организации (ISTQB): часть 2

Процессный — опирается на стандарты и регламентированные процессы.

Чек-листный — использует заранее подготовленные перечни проверок.

Эвристический — основан на опыте и суждениях тестирующего.

На практике подходы комбинируют: для критичных сервисов — риск-ориентированный, для рутинных — чек-листный.

Тестовая стратегия: сквозной пример

Разграничение уровней документации

Тестовая политика — организационный уровень: цели и правила для всей компании.

Тестовая стратегия — уровень программы или проекта: подход и приоритеты.

Тестовый план — уровень работы: способ, роли, сроки, критерии.

Стратегия для API создания клиента — `POST /api/v1/clients/create-individual`

Объект тестирования: сервис создания клиента.

Подход: риск-ориентированный, дополнен аналитическим анализом требований.

В области тестирования: валидация обязательных полей и форматов групп `contacts / individual / tax`.

Вне области тестирования: проведение платежей — отдельный сервис с иным риск-профилем.

Обоснование: заведение клиента — критический путь, с него начинается процессинговая цепочка.

Уровни проверок: функциональные проверки API; интеграционные — по отдельному плану.

Для практики 2 стратегия составляется на всю систему — 2–3 абзаца и перечень уровней проверок: что входит в область тестирования, что вне её и на каком основании.

Тестовый план

Уровни плана

Проектный — на весь проект.

Релизный — на конкретный релиз.

Итерации — на итерацию или веху.

Разделы плана (ISO/IEC/IEEE 29119-3)

Область тестирования и подход.

Критерии входа и выхода; критерии приостановки и возобновления.

Ресурсы, расписание, метрики, риски.

Сквозная иллюстрация — пункты плана для задачи «Создание клиента»

Область: валидация и дубли групп contacts / individual / tax

Критерий входа: стенд доступен, требования согласованы

Приостановка / возобновление: при дефекте критического уровня проверки приостанавливаются до его устранения

Подход: чёрный ящик; КЭ и ГЗ; риск-ориентированный приоритет

Критерий выхода: 0 дефектов критического и высокого уровней; ≥ 95 % покрытия ключевых полей

Ресурсы и риски: 1 тестировщик, 1 аналитик; риск — поздние правки схемы валидации

Стандарты: IEEE 829 отозван; актуальная линейка шаблонов тест-планов — ISO/IEC/IEEE 29119-3.

Чек-листы

Чек-лист — упорядоченный перечень проверок, определяющий, что подлежит проверке, без конкретных входных данных, шагов и ожидаемых результатов. Разновидность высокоуровневого набора проверок.

Свойства

Краткость формулировок · пригодность к повторному применению · логическая полнота без дублирования. Ожидаемые результаты обычно не приводятся.

Отличие от тест-кейса

Чек-лист определяет, что проверяется; тест-кейс — как: данные, шаги, ожидаемый результат. Соответствует высокоуровневому тест-кейсу.

Сквозной пример — чек-лист «Создание клиента»

- ☐ Заполненность обязательных полей групп contacts / individual / tax
- ☐ Допустимость отсутствия необязательных полей
- ☐ Соответствие форматов полей email, phone, birth_date, tax_number
- ☐ Отклонение дублей external_id

Области применения: дымовое тестирование · критический путь · каркас исследовательского тестирования.

Тест-кейс: структура и виды

Тест-кейс (test case) — совокупность входных данных, предусловий, шагов и ожидаемых результатов, разработанная для проверки конкретной цели. В отличие от чек-листа содержит конкретику и воспроизводим.

Классические поля

Идентификатор · предусловие · шаги · ожидаемый результат · статус.

Дополнительно: приоритет · связанное требование · постусловие.

Уровни и виды

Высокоуровневый — без конкретных значений; **низкоуровневый** — с данными и шагами.

Позитивные — валидные классы; **негативные** — невалидные.

Правило комбинирования: несколько валидных значений допустимо объединять в один тест-кейс; несколько невалидных объединять нельзя, иначе неоднозначно, какая проверка вызвала отказ. Критерий качества: отсутствие входных данных, ожидаемого результата или цели делает тест-кейс непригодным.

Тест-кейс: пример

Негативный тест-кейс по граничному значению — TC-CLI-014

Объект: POST /api/v1/clients/create-individual · проверяемое условие: длина поля email

Поле тест-кейса	Значение
Идентификатор	TC-CLI-014
Связанное требование	email длиной не более 254 символов и корректного формата
Вид и уровень	негативный, низкоуровневый; граничное значение
Предусловие	стенд доступен; клиент с указанным email не создан
Шаги	выполнить POST /api/v1/clients/create-individual; email длиной 255 символов; остальные поля валидны
Ожидаемый результат	HTTP 400, код VALIDATION_ERROR, клиент не создан
Постусловие	запись клиента отсутствует в хранилище

Пример из практики

Фрагмент тестового плана · задача «Создание клиента»

Область тестирования: POST /api/v1/clients/create-individual — валидация и дубли групп contacts / individual / tax.

Критерий входа: стенд доступен, требования согласованы, тестовые данные подготовлены.

Приостановка / возобновление: при блокирующем дефекте критического уровня проверки приостанавливаются до его устранения.

Риски: поздние правки схемы валидации → повторный прогон набора.

Формулировки иллюстративные: демонстрируют структуру и уровень детализации рабочего документа.

Подход: чёрный ящик; классы эквивалентности и граничные значения; риск-ориентированный приоритет.

Критерий выхода: отсутствие открытых дефектов критического и высокого уровней; ≥ 95 % покрытия обязательных полей.

Ресурсы: 1 тестировщик, 1 аналитик; окружение dev; 3 итерации.

Метрики: доля выполненных тест-кейсов, число открытых дефектов по уровням.

Практика 2: что создать

01

Тестовая стратегия СМП

Цель тестирования, уровни проверок, обоснование исключений из области.

02

Тестовый план СМП

Область и подход, критерии входа и выхода, ресурсы, расписание, риски.

03

Test-design ядра

Классы эквивалентности, границы и pairwise для Authorization и Card-Management; модель и набор PICT.

04

Чек-листы

Дымовое тестирование и критический путь для остальных сервисов.

Резюме лекции 3

Тест-дизайн — систематический выбор информативного подмножества проверок из неограниченного множества; техника задаёт правило выбора.

Классы эквивалентности: по одному представителю на класс; валидные и невалидные классы проверяются отдельно.

Граничные значения: три точки на каждую границу — значение на границе, ниже и выше неё.

Попарное тестирование: покрытие всех пар значений параметров вместо полного перебора комбинаций; набор строится инструментами построения попарных наборов.

Тестовая документация — результат: стратегия → план → чек-листы → тест-кейсы.

Связка «техника → тест-кейс»: техника определяет, какие значения выбираются; тест-кейс фиксирует данные, шаги и ожидаемый результат.

Практика 2 (18.09): проектная тестовая документация по СМП — стратегия, тестовый план, test-design ядра (классы, границы, pairwise) с тест-кейсами и чек-листы; артефакт 2.

МОДУЛЬ 2 · ПРАКТИКА 2

Тест-дизайн: документация проекта

Охват: три слоя документации



Проект: вся система

Стратегия и тестовый план — обоснование области и приоритетов для всех сервисов СМП.



Ядро: Authorization и Card-Management

Полный test-design: классы эквивалентности, границы и pairwise.



Периферия: прочие сервисы

Чек-листы дымового тестирования и критического пути; классы верхнего уровня.

Артефакт 2: состав

№	Документ	Слой	Файл
1	Тестовая стратегия СМП	проект	docs/practice-2/test-strategy.md
2	Тестовый план СМП	проект	docs/practice-2/test-plan.md
3	Test-design ядра: классы эквивалентности, границы, pairwise	ядро	docs/practice-2/test-design.md
4	Чек-листы: дымовое тестирование и критический путь	периферия	docs/practice-2/checklists.md
5	Модель PICT и сгенерированный попарный набор	ядро	docs/practice-2/pict/

Инструмент: PICT — модель и запуск

Запуск

```
pict /?  
pict model.txt > cases.txt
```

Требования к модели

- не менее шести параметров;
- по два-четыре значения на параметр;
- ограничения модели (constraints);
- на выходе — покрытие каждой пары значений.

PIST: модель авторизации

```
card_status: ACTIVE, INACTIVE, BLOCKED, EXPIRED
amount_vs_daily: below, equal, above
amount_vs_monthly: below, equal, above
amount_vs_balance: below, equal, above
expiry: valid, current_month, expired
terminal_type: pos, atm, ecom
mcc: grocery, restaurant, electronics, travel

# Ограничения
IF [card_status] <> "ACTIVE" THEN [amount_vs_daily] = "below"
    AND [amount_vs_monthly] = "below" AND [amount_vs_balance] = "below";
IF [expiry] = "expired" THEN [amount_vs_balance] = "below";
```

Полный перебор — $4 \times 3 \times 3 \times 3 \times 3 \times 4 = 3888$ комбинаций; попарный набор покрывает все пары при кратном сокращении числа строк.

Ядро: распределение техник

Объект	Техники	Уровень детализации
Authorization	Классы эквивалентности, границы, pairwise	глубокий
Card-Management	Классы эквивалентности, границы (Luhn, expiryDate MMYU)	глубокий
Gateway, Switch	Классы маршрутизации, границы лимита запросов, обработка 503	чек-лист
Симуляторы, Logger, Dashboard	Классы сценариев, дымовые проверки	чек-лист

Тест-кейс: поля и правила

Обязательные поля

Идентификатор и связанное требование

Вид: позитивный или негативный

Предусловие, шаги, ожидаемый результат

Источник: класс, граница или строка попарного набора

Правила

Один невалидный класс — один тест-кейс

Несколько валидных значений допустимо объединять

Каждая строка попарного набора — отдельный тест-кейс

Тест-кейсы размещаются в составе документа test-design

Оценка: 7 баллов

Компонент	Балл
Тестовая стратегия СМП	1
Тестовый план СМП	1
Чек-листы: дымовое тестирование и критический путь	1
Классы эквивалентности	1
Граничные значения	1
Pairwise: модель, ограничения, маппинг в тест-кейсы	2
Итого	7

Источники

ISTQB — Certified Tester Foundation Level (CTFL) v4.0, Syllabus и Glossary.

Copeland, Lee. *A Practitioner's Guide to Software Test Design*. Artech House.

Куликов, С. С. *Тестирование программного обеспечения*. Базовый курс.

Beizer, Boris. *Software Testing Techniques* — парадокс пестицидов.

Kaner, Bach, Pettichord. *Lessons Learned in Software Testing* (LLST / TTS).

Kuhn, Wallace — NISTIR 6732, 2001; **Kuhn, Reilly** — NASA/IEEE, 2002.

ISO/IEC/IEEE 29119 — документация тестирования; исторический ориентир — IEEE 829.



Вопросы?